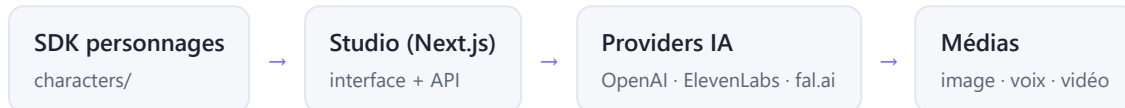


Virtual Humans Studio

Comment fonctionne l'application, module par module : le SDK des personnages virtuels, le Studio Next.js de génération, et les fournisseurs d'IA (image, voix, vidéo).



Projet : `virtual-humans` — Personnages actuels : **Mei & Tom** · Objectif : vidéos publicitaires (micro-trottoir 60 s) pour promouvoir des apps.

1 Vue d'ensemble & concepts clés

Deux briques séparées

SDK (`characters/`) = l'identité complète d'un personnage virtuel (fiches, mémoire, assets).
Studio (`studio/`) = l'appli Next.js qui lit le SDK et génère les médias. On ne mélange jamais les deux.

Personnage actif

Sélecteur global en haut du menu. Toutes les pages (image, voix, vidéo...) travaillent sur ce personnage. Le choix est mémorisé dans le navigateur.

`characterId` vs `characterCode`

`characterId` = identifiant technique (URL/API, ex. `tom`).
`characterCode` = code métier lisible (ex. `TOM-001`). Les deux sont **uniques** : deux packages ne peuvent pas les partager.

Identité verrouillée

Pour garder le même visage d'un plan à l'autre : une **image de départ** (photo du perso ou décor généré) sert de 1re frame ; la vidéo se contente de l'**animer**. Pas d'invention de visage.

Le flux type : on choisit le personnage → on prépare une image (tenue/décor) → on anime en vidéo (muette) → on ajoute la voix par lip-sync → on assemble les plans en une pub de 60 s. Chaque génération est estimée en \$ avant l'appel.

2 Les modules de l'interface (menu de gauche)

Tableau de bord

/ (page.tsx)

Vue d'accueil du personnage actif : cartes Identité, Apparence, Personnalité, Voix. Indicatif — le pilotage se fait dans les Studios.

Studio Scène

/scene

Compose une présentation : tenue + lieu + pitch + produit. Bloc **Décor** (PuLID Flux) place le perso dans un lieu, puis Kling l'anime (identité verrouillée). Compteur de temps pendant la génération.

Studio Image

/image

Génère des images via OpenAI (gpt-image-1) : taille, qualité, quantité. Sert de bibliothèque de références par personnage.

Studio Vidéo

/video

Génération vidéo unitaire. Choix du modèle (Veo, Seedance, Kling, Runway, MiniMax), durée et ratio adaptés au modèle. File d'attente + suivi de statut.

Storyboard 60 s

/storyboard

Enchaîne 7 plans (preset micro-trottoir). Kling image→vidéo pour l'identité, **Décor par plan** (un lieu différent, visage conservé), puis **assemblage FFmpeg** en une vidéo de ~60 s.

Réglages

/settings

État des clés API (OpenAI, ElevenLabs, fal.ai), modèle de `.env.local`, source du SDK et tarifs voix.

Personnages

/characters · /characters/[id]

Liste des packages détectés + santé de chacun. La fiche détaille tenues, identité, poses, expressions, voix, capacités, limites, 11 mémoires, jauges de personnalité, phrases candidates et **alertes qualité** (assets manquants, collisions).

Produits / Apps

/products

Bibliothèque des apps à promouvoir : nom, pitch, couleur, URL et **captures d'écran** (ajout/suppression). Réutilisées dans Studio Scène.

Studio Voix

/voice

Synthèse vocale ElevenLabs. Chaque personnage a **sa propre voix** (`voice/config.json` : Mei et Tom ont des voiceld distincts).

Studio Lip-sync

/lipsync

Synchronise une voix sur une vidéo muette (VEED ou Sync v3). C'est l'étape qui fait « parler » le personnage face caméra.

Budget

/budget

Détail des dépenses estimées (par type/modèle) + total. Bouton **reset** (aussi présent sur la carte du menu) pour repartir de zéro par campagne.

Modales de confirmation

components/confirm.tsx

Remplace les pop-ups du navigateur. Hook `useConfirm()` → modale stylée (Échap = annuler, Entrée = valider), variante rouge pour les suppressions.

3 Le runtime « personnage » (le cerveau du SDK)

Dossier `studio/src/runtime/character/` — lit un package sur disque, le valide strictement, et l'expose via une API versionnée. Aucun repli silencieux : en cas de problème, une erreur typée explicite.

loader.ts

Charge un package depuis le disque, valide, remonte les problèmes non bloquants (assets manquants...) et invalide les assets marqués.

registry.ts

Recense tous les personnages et applique la **garde d'unicité** : collision de `characterId` / `characterCode` → erreur (HTTP 409), pas de choix arbitraire.

errors.ts

Hiérarchie d'erreurs : NotFound, PackageInvalid, DocumentMissing, IdentityAssetMissing, DuplicateId/Code...

schema.ts (Zod)

Schémas stricts : identité, tenues, voix, personnalité, mémoires, qualité des données. Type-safe, échec explicite si malformé.

personality.ts

Extrait la personnalité du Markdown (`02_PERSONALITY.md`) : traits, jauges, style, phrases candidates.

http.ts + api/v1/characters

Mapping pur lookup → statut/corps HTTP. Routes `/api/v1/characters` et `/api/v1/characters/[id]` (testables, 200/404/409).

4 Fournisseurs IA & modèles (prix indicatifs)

Usage	Fournisseur / modèle	Mode	Prix indicatif *	Notes
Image	OpenAI <code>gpt-image-1</code>	texte→image	0,011–0,25 \$ / image	Selon taille & qualité
Décor / still identité	fal <code>flux-pulid</code> (PuLID Flux)	réf.→image	~0,05 \$ / image	Préserve le visage ; sert de 1re frame
Voix	ElevenLabs TTS	texte→audio	~0,15 \$ / 1k car.	1 voix par personnage
Vidéo	Veo 3.1 Fast	texte→vidéo	~0,40 \$/s	voix+lèvres natives 9:16, 16:9
Vidéo	Kling 2 Master	image→vidéo	~0,28 \$/s	défaut Scène/Storyboard ; identité par 1re frame ; muet
Vidéo	Seedance 2.0	réf.→vidéo	~0,15 \$/s	modéré refuse les visages photoréalistes
Vidéo	Runway Gen-3 Turbo	image→vidéo	~0,05 \$/s	muet
Vidéo	MiniMax Hailuo 02	texte→vidéo	~0,05 \$/s	muet
Lip-sync	VEED / Sync v3	vidéo+voix	0,40 \$ / 8 \$ /min	éco vs haute fidélité
Montage	fal <code>ffmpeg-api/merge-videos</code>	assemblage	négligeable	concatène les plans

* Prix publics indicatifs, ajustables via variables d'environnement (`FAL_KLING_USD_PER_SEC` , `ELEVENLABS_USD_PER_1K_CHARS` ...). Toujours vérifier les tarifs réels chez le fournisseur. Providers : `lib/providers/openai-image.ts` , `elevenlabs-voice.ts` , `fal.ts` .

5 Librairies internes (studio/src/lib)

sdk.ts

Pont vers le disque : liste personnages, assets, tenues, produits, config voix. Accepte un `character`.

pricing.ts

Catalogue des modèles + fonctions d'estimation (\$) pour image, voix, vidéo, lip-sync, décor, montage.

budget.ts

Journal des dépenses persisté dans `.data/spend.json` : `addSpend`, `resetSpend`, résumé.

client.ts

Helpers front : `apiGet/apiPost`, `usd()`, `withCharacter()`, événement `budget:refresh`.

character-context.tsx

Contexte React du personnage actif (persisté en `localStorage`).

media-store.ts + reflib.ts

Dernières images/vidéos et bibliothèque de références, **par personnage** (`localStorage` namespacé).

assemble.ts

Logique d'assemblage des plans (montage 60 s).

providers/*

Adaptateurs OpenAI, ElevenLabs, fal.ai (upload, file d'attente, statut).

6 Principales routes API (studio/src/app/api)

Route	Rôle
<code>/api/characters</code> , <code>/api/character</code>	Liste des personnages / détail du personnage actif
<code>/api/assets</code> , <code>/api/asset</code> , <code>/api/outfits</code>	Liste et service des assets (identité, poses, tenues...)
<code>/api/products</code> , <code>/api/product-screen</code>	CRUD produits + captures (ajout / service / suppression)
<code>/api/generate/image</code>	Génération d'image (OpenAI)
<code>/api/generate/voice</code>	Synthèse vocale (ElevenLabs, voix du personnage)
<code>/api/generate/scene-image</code>	Image de décor identité-préservée (PuLID Flux)
<code>/api/generate/video</code> + <code>/status</code>	Lance la vidéo (fal) et suit la file d'attente
<code>/api/generate/lipsync</code>	Synchronisation labiale voix ↔ vidéo
<code>/api/generate/merge</code>	Assemblage des plans (FFmpeg)
<code>/api/budget</code>	GET résumé · DELETE reset du cumul
<code>/api/estimate</code> , <code>/api/video-models</code> , <code>/api/settings</code>	Estimations, catalogue modèles, état config
<code>/api/v1/characters[/id]</code>	API runtime versionnée (validation stricte, 409 si collision)

7 Recette : produire une pub micro-trottoir de 60 s

- 1 **Choisir le personnage** actif (Mei ou Tom) dans le menu de gauche.
- 2 **Préparer le produit** dans « Produits / Apps » : nom, pitch, captures d'écran de l'app.
- 3 **Ouvrir Storyboard 60 s** : les 7 plans du preset micro-trottoir sont pré-remplis.
- 4 **Frame de départ** : sélectionner une photo d'identité (verrouille visage + tenue).
- 5 **Décor par plan** (optionnel) : saisir un lieu (« rue animée »...) puis « Générer le décor » → le perso est placé dans le décor, identité préservée.
- 6 **Générer les plans** (Kling image→vidéo, muet) — ~3–4 min pour 5 s, c'est normal.
- 7 **Voix + lip-sync** : passer chaque plan (ou le montage) dans Studio Lip-sync pour faire parler le personnage.
- 8 **Assembler** les plans en une vidéo de ~60 s (FFmpeg merge).
- 9 **Surveiller le budget** : chaque étape s'ajoute au cumul ; **reset** en début de campagne.

8 Où sont stockées les données

characters/<Nom> SDK v1.0.0/

Le package personnage : fiches Markdown (identité, personnalité...), `memory/`, `prompts/`, `voice/config.json`, `assets/` (identity, expressions, poses, outfits).

products/

Bibliothèque des apps à promouvoir : métadonnées + captures d'écran (à la racine du repo).

studio/.data/spend.json

Journal des dépenses estimées (cumul, reset possible).

Navigateur (localStorage)

Personnage actif, dernières images/vidéos et bibliothèque de références — **par personnage**.

Clés API dans `studio/.env.local` : `OPENAI_API_KEY`, `ELEVENLABS_API_KEY`, `ELEVENLABS_VOICE_ID`, `FAL_KEY`.
Redémarrer `npm run dev` après modification.